

Recursividade

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

- Um procedimento ou função recursiva é aquele que contém em sua descrição uma ou mais chamadas a si próprio.
- A chamada é dita chamada recursiva.
- O procedimento recursivo inicia sua execução a partir de uma chamada dita externa.
- Um procedimento ou função é dito não recursivo quando é sempre executado a partir de uma chamada externa.
- Toda função recursiva corresponde a uma outra função não recursiva que executa a mesma computação.

Funções recursivas

- Precisam de uma condição para finalizar as chamadas recursivas, para evitar uma execução sem fim.
- Chamadas recursivas em uma linguagem de programação utilizam uma pilha de execução.
 - A pilha é criada pelo próprio programa em sua área de memória (no *heap*).
 - A cada chamada recursiva os valores das variáveis locais da função na nova chamada são inseridos em nova posição na pilha de execução.
 - Quando cada chamada recursiva termina uma posição da pilha (sempre no topo) é desempilhada.

Algoritmo Iterativo - Fatorial

```
function Fat(n)
    f = 1
    for i = 2 to n do
        f = i * f
    return f
```

```
var
    int c, f

    read(c)
    f = Fat(c)
    print(f)
```

Fatorial Iterativo - Python

```
print("Fatorial")
n=int(input("Digite o numero para calculo do fatorial:"))
fat = 1
for i in range(n,1,-1):
    fat = fat * i
print(fat)
```

Fatorial Iterativo – programa C

```
# include <stdio.h>
```

```
int fat(int n)
```

```
{
```

```
    int i;
```

```
    int f;
```

```
    f = 1;
```

```
    for (i= 2;i<= n;++i)
```

```
        f = i * f;
```

```
    return f;
```

```
}
```

```
int main (void)
```

```
{
```

```
    int c;
```

```
    printf("Digite o valor:");
```

```
    scanf("%d",&c);
```

```
    printf("%d",fat(c));
```

```
    return 0;
```

```
}
```

Algoritmo Recursivo - Fatorial

```
function Fat(n)
    if n > 0 then
        fat = n * Fat(n-1)
    else
        fat = 1
    return fat
```

```
var
    int c, f
begin
    read(c)
    f := Fat(c)
    print(f)
end
```

Fatorial Recursivo – programa C

```
# include <stdio.h>
```

```
int fat(int n)
```

```
{
```

```
    if (n==0)
```

```
        return 1;
```

```
    else
```

```
        return n*fat(n-1);
```

```
}
```

```
int main (void)
```

```
{
```

```
    int c;
```

```
    printf("Digite o valor:");
```

```
    scanf("%d",&c);
```

```
    printf("%d",fat(c));
```

```
    return 0;
```

```
}
```

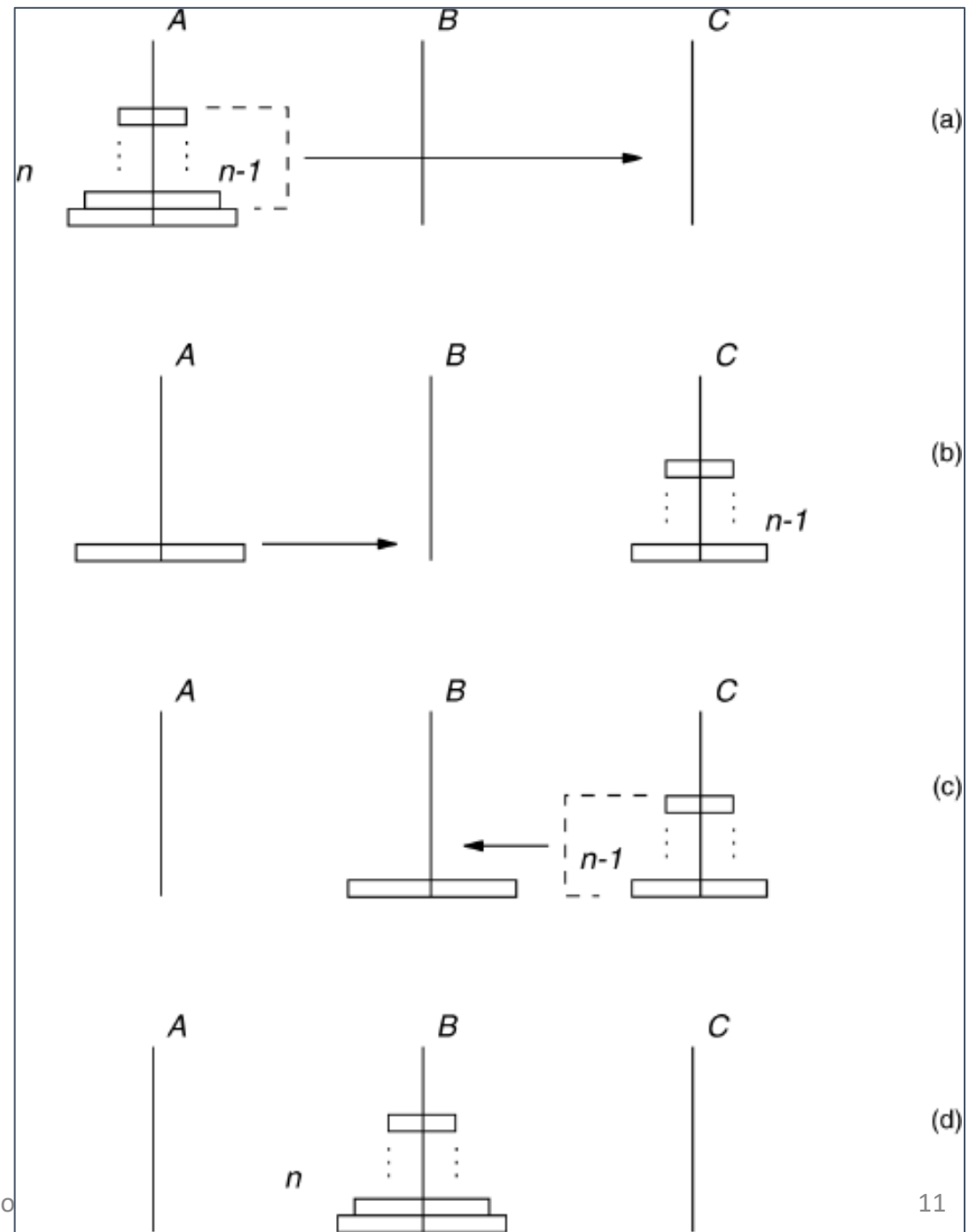

Fatorial Recursivo – programa Python

```
def fatorial( n):  
    if n <1:  
        return 1  
    else:  
        return n * fatorial( n - 1) #chamada recursiva  
  
n = int(input("Digite um valor para cálculo fatorial:") )  
print ('O fatorial:', fatorial( n))
```

Torres de Hanoi

- Temos 3 pinos identificados como A, B e C, e n discos de diâmetros diferentes.
- Inicialmente todos os discos estão empilhados no pino A, em ordem decrescente de tamanho, de baixo para cima.
- O objetivo é empilhar todos os discos no pino B, sendo que:
 - Apenas um disco pode ser movido por vez;
 - Qualquer disco não pode ser colocado sobre outro de tamanho menor.

- Mover n pinos de A para B será:
 - Mover $n-1$ pinos de A para C;
 - mover 1 pino de A para B;
 - mover $n-1$ pinos de C para B.
 - Solução recursiva.
- Como mover $n-1$ pinos de A para C e depois de C para B?
 - Mover $n-1$ pinos de A para C:
 - Mover $n-2$ pinos de A para B;
 - Mover o pino que sobra de A para C;
 - Mover $n-2$ pinos de B para C.



- Procedimento recursivo Hanoi:

Function hanoi(n, A, B, C)

if $n > 0$ then

hanoi(n-1, A, C, B)

mover o disco do topo da torre A para o topo da torre B

hanoi(n-1, C, B, A)

Torre de Hanoi

```
def moveTower(height, fromPole, toPole, withPole):  
    if height >= 1:  
        moveTower(height-1,fromPole,withPole,toPole)  
        moveDisk(fromPole,toPole)  
        moveTower(height-1,withPole,toPole,fromPole)  
  
def moveDisk(fp,tp):  
    print("Movendo o disco de", fp, "para", tp)  
  
moveTower(3,"A","B","C")
```